

Assignment 4: Information Extraction (Regex, Edit Distance, NER)

Learning Objectives

- Extract structured information with **regular expressions**
- Compute **Levenshtein (edit) distance** for fuzzy matching
- Try **Named Entity Recognition (NER)** baselines for Chinese

Tasks (TODO)

- [1] Use Regular Expressions to extract dates, phone numbers, and email addresses.
請利用正規表示法擷取日期、電話、Email
- [2] Use a spaCy model to identify personal names (PERSON) and geographic locations (GPE/LOC).
請利用spacy模型找到人名與地名
- [3] Use Levenshtein Distance to align entities with a standard list and replace the inconsistent terms in the original text.
請利用Levenshtein 找出標準名單，並且代換原本混亂文章的文字。

(所有程式運算請截圖來呈現結果，請繳交PDF檔案)

0. Setup

```
In [ ]: # !pip install regex python-Levenshtein spacy jieba transformers torch
# Optional Chinese NER models can be used via Hugging Face transformers.
```

[1]Regex Extraction Practice

```
In [ ]: TEXT = """
王小明於2025年8月15日到台北市中山區松江路123號拜訪陳小姐，
聯絡電話為(02) 2345-6789，Email: test@example.com。
"""
print(TEXT)
```

Please write the regular expression to get the following results:

```
Dates: [('2025', '8', '15')]
Phones: ['(02) 2345-6789']
Emails: ['test@example.com']
```

```
In [ ]: import re

PAT_DATE = re.compile(r'(\d{4})年(\d{1,2})月(\d{1,2})日')
PAT_PHONE = re.compile(r'\(\d{2}\)\s*\d{4}-\d{4}')
PAT_EMAIL = re.compile(r'[\w.+-]+@[ \w-]+(?:\. [\w-]+)+')

print('Dates:', PAT_DATE.findall(TEXT))
print('Phones:', PAT_PHONE.findall(TEXT))
print('Emails:', PAT_EMAIL.findall(TEXT))
```

終端機執行結果：

```
python week08/task1_regex.py

=====
Task 1: Regex Extraction
=====
原文： 王小明於2025年8月15日到台北市中山區松江路123號拜訪陳小姐，
聯絡電話為(02) 2345-6789，Email: test@example.com。

Dates : [('2025', '8', '15')]
Phones: ['(02) 2345-6789']
Emails: ['test@example.com']

~/De/nlp_homework / on main ?6
```

Observation for Task 1

日期的 regex 使用三個 capture groups，所以 `findall()` 會回傳 `('2025', '8', '15')` 這種 tuple，而不是整段日期字串。電話號碼中的左右括號在 regex 裡有特殊意義，因此需要用 `\(` 和 `\)` 轉義。Email pattern 這裡採用作業範例足夠使用的版本，沒有刻意寫成完整 RFC 規格，因為本題目標是從一般文字中抓出常見 email 格式。

[2]NER easy Practice

如果第一次執行時找不到 `zh_core_web_sm`，請先在終端機或 notebook 執行：

```
python -m spacy download zh_core_web_sm
```

```
In [ ]: sample_text = """
「昨晚八點，鴻海精密的前董事長郭台明在台北信義區舉辦了一場科技論壇。
會中他提到了蘋果公司的創辦人賈伯斯對全球產業的影響。
與此同時，蔡英蚊也在臉書上轉發了相關新聞。
雖然論壇地點原本預計在香港的銅羅灣舉行，但因為行程調整，最後改到了台灣。
不少來自美國加洲的工程師也透過視訊參與了這場由聯合國教科文組織贊助的活動，
時間定在2024年三月正式發布報告。」
"""
```

```
In [ ]: import spacy

# Download first if needed: python -m spacy download zh_core_web_sm
try:
```

```

nlp = spacy.load("zh_core_web_sm")
except OSError as exc:
    raise OSError(
        "spaCy Chinese model zh_core_web_sm is not installed. "
        "Run: python -m spacy download zh_core_web_sm"
    ) from exc

# 使用 A4 作業提供的 sample_text
doc = nlp(sample_text)

```

```

In [ ]: # 練習：篩選出所有的「人名 (PERSON)」與「地名 (GPE/LOC)」
people = [ent.text for ent in doc.ents if ent.label_ == "PERSON"]
locations = [ent.text for ent in doc.ents if ent.label_ in {"GPE", "LOC"}]

print("所有辨識到的實體：")
for ent in doc.ents:
    print(f" {ent.text}\t{ent.label_}\t({spacy.explain(ent.label_)})")

print(f"\n找到的人名：{people}")
print(f"找到的地名：{locations}")

```

終端機執行結果：

```

~/De/nlp_homework / on main ?6 python week08/task2_ner.py

=====
Task 2: spaCy Chinese NER
=====
所有辨識到的實體：
  昨晚          TIME      (Times smaller than a day)
  郭台明        PERSON    (People, including fictional)
  台北          GPE       (Countries, cities, states)
  蔡英蚊        PERSON    (People, including fictional)
  香港          GPE       (Countries, cities, states)
  聯合國教      ORG       (Companies, agencies, institutions, etc.)
  2024年三月    DATE      (Absolute or relative dates or periods)

-----
找到的人名 (PERSON) : ['郭台明', '蔡英蚊']
找到的地名 (GPE/LOC): ['台北', '香港']

```

[3]Edit Distance (Levenshtein)

Task: Entity Normalization and Text Cleaning

Background

In real-world Information Extraction (IE) pipelines, raw text often contains "noisy" entities. Even after identifying entities using Named Entity Recognition (NER) or Regular Expressions, you will frequently encounter variations of the same entity due to:

- **Typos:** e.g., "Google" (with a capital 'l') vs. "Google".
- **Abbreviations:** e.g., "TSMC" vs. "Taiwan Semiconductor Manufacturing Company".
- **Inconsistent Naming:** e.g., "National Taiwan University" vs. "NTU".

To create a clean database, we must **normalize** these variations into a **Standard Golden List**.

Objectives

1. **Identify** potential entities within a messy text using spaCy or Regex.
2. **Define** a `standard_list` of canonical entity names.
3. **Implement** a fuzzy matching algorithm using **Levenshtein Distance** to align messy entities with the standard names.
4. **Transform** the original text by replacing all messy variants with their standardized versions.

Instructions (TODO)

1. 請列出標準名稱表：**Define your Standards:** Create a `standard_list` containing the 4 official names you want to keep (e.g., "Taiwan Semiconductor Manufacturing Company", "Hon Hai Precision Industry", "Google Alphabet", "National Taiwan University").
2. 請計算相似度：**Calculate Similarity:** Use `Levenshtein.ratio()` to compare each item in the `entity_list` against your `standard_list`.
3. 請設定相似度的閾值 **Threshold Matching:**
 - Set a similarity **threshold** (e.g., 0.6).
 - If the similarity score is above the threshold, map the messy entity to the best-matching standard name.
4. 請進行自動替換：**Automatic Replacement:** Write a loop or function to iterate through the `messy_text` and replace every occurrence of a "noisy" entity with its "standard" counterpart.

Requirements

- Display the final "cleaned" version of the text.
- 請將過程截圖，並且呈現最後按照標準名單進行自動替換過後的結果文章。

Try `RapidFuzz` for calculating Levenshtein for Chinese. When we calculate the Levenshtein distance, we regard the Chinese string character by character.

```
In [ ]: !pip install rapidfuzz
```

```
In [ ]: entity_list = [  
    "台積電", "台灣積體電路製造公司", "台積電股份有限公司", "台機電",  
    "鴻海", "鴻海精密", "鴻海科技集團", "紅海精密",  
    "Google", "GoogIe", "Google Inc.", "谷歌",  
    "國立臺灣大學", "台灣大學", "台大", "國立台灣大學"  
]
```

```
messy_text = ""
```

根據今日財報分析，台灣積體電路製造公司 在本季表現亮眼，
而另一家半導體大廠 台機電 也表示產能滿載。
與此同時，鴻海精密 與其夥伴 鴻海科技集團 宣布了新的合作計畫，
但在網路論壇上，部分網友卻誤植為 紅海精密。

學術界方面，國立臺灣大學 發表了最新的 AI 研究成果，
參與研究的學生不少來自 台灣大學 與 台大，
這項研究也獲得了美國科技巨頭 GoogIe 的技術支援，
雖然在合約中對方的正式名稱標註為 Google Inc.。
""

```
In [ ]: from rapidfuzz.distance import Levenshtein

# Two ways to calculate similarity: Levenshtein distance and similarity ratio
str1 = "台積電"
str2 = "台機電"

# 1. 編輯距離 (Distance)
# 數值代表需要經過幾次「增、刪、改」才能變成另一個字串
dist = Levenshtein.distance(str1, str2)
print(f'distance("{str1}", "{str2}") = {dist}')

# 2. 相似度比例 (Normalized similarity)
# rapidfuzz 的 API 為 normalized_similarity，回傳 0~1，乘以 100 即百分比
ratio = Levenshtein.normalized_similarity(str1, str2) * 100
print(f'similarity("{str1}", "{str2}") = {ratio:.2f}')
```

```
In [ ]: import re
from rapidfuzz.distance import Levenshtein

standard_list = [
    "台灣積體電路製造股份有限公司",
    "鴻海精密工業股份有限公司",
    "Google LLC",
    "國立臺灣大學",
]

# 縮寫、俗稱、跨語言別名 Levenshtein 分數會偏低，
# 所以額外維護一張 alias 表作為 fallback。
alias_to_standard = {
    "台積電": "台灣積體電路製造股份有限公司",
    "台灣積體電路製造公司": "台灣積體電路製造股份有限公司",
    "台積電股份有限公司": "台灣積體電路製造股份有限公司",
    "台機電": "台灣積體電路製造股份有限公司",
    "鴻海": "鴻海精密工業股份有限公司",
    "鴻海精密": "鴻海精密工業股份有限公司",
    "鴻海科技集團": "鴻海精密工業股份有限公司",
    "紅海精密": "鴻海精密工業股份有限公司",
    "Google": "Google LLC",
    "GoogIe": "Google LLC",
    "Google Inc.": "Google LLC",
    "谷歌": "Google LLC",
    "國立臺灣大學": "國立臺灣大學",
}
```

```

    "台灣大學": "國立臺灣大學",
    "台大": "國立臺灣大學",
    "國立台灣大學": "國立臺灣大學",
}

def best_levenshtein_match(entity, standards):
    scores = [
        (standard, Levenshtein.normalized_similarity(entity, standard))
        for standard in standards
    ]
    return max(scores, key=lambda item: item[1])

def normalize_entity(entity, standards, threshold=0.6):
    best_standard, best_score = best_levenshtein_match(entity, standards)
    if best_score >= threshold:
        return best_standard, best_score, "Levenshtein"
    if entity in alias_to_standard:
        alias_standard = alias_to_standard[entity]
        alias_score = Levenshtein.normalized_similarity(entity, alias_standard)
        return alias_standard, alias_score, "alias"
    return None, best_score, "skip"

threshold = 0.6
entity_mapping = {}

print("標準名稱表:")
for standard in standard_list:
    print(" -", standard)

print("\n別名 → 標準對應結果:")
for entity in entity_list:
    standard, score, method = normalize_entity(entity, standard_list, threshold)
    if standard is not None:
        entity_mapping[entity] = standard
    print(f" {entity:<11} → {standard}    [score={score:.2f}, {method}]")

# 用單次 re.sub 一次掃完原文，避免重複替換的問題：
# 例如 鴻海精密 → 鴻海精密工業股份有限公司 之後，若再用 .replace 處理 鴻海，
# 會把標準名稱開頭的「鴻海」再替換一次，產生「鴻海精密工業股份有限公司精密工業股份有限公司」。
# alternation 用長度由長到短排序，確保 鴻海精密 比 鴻海 優先匹配。
pattern = re.compile(
    "|".join(re.escape(alias) for alias in sorted(entity_mapping, key=len, reverse=True))
)
cleaned_text = pattern.sub(lambda m: entity_mapping[m.group(0)], messy_text)

print("\n清理後文章:")
print(cleaned_text)

```

終端機執行結果：

```
~/De/nlp_homework / on main ?7 python week08/task3_normalize.py

=====
Task 3: Entity Normalization with Levenshtein Distance
=====

標準名稱表：
- 台灣積體電路製造股份有限公司
- 鴻海精密工業股份有限公司
- Google LLC
- 國立臺灣大學

別名 → 標準對應結果：
台積電 → 台灣積體電路製造股份有限公司 [score=0.21, alias]
台灣積體電路製造公司 → 台灣積體電路製造股份有限公司 [score=0.71, Levenshtein]
台積電股份有限公司 → 台灣積體電路製造股份有限公司 [score=0.64, Levenshtein]
台機電 → 台灣積體電路製造股份有限公司 [score=0.14, alias]
鴻海 → 鴻海精密工業股份有限公司 [score=0.17, alias]
鴻海精密 → 鴻海精密工業股份有限公司 [score=0.33, alias]
鴻海科技集團 → 鴻海精密工業股份有限公司 [score=0.17, alias]
紅海精密 → 鴻海精密工業股份有限公司 [score=0.25, alias]
Google → Google LLC [score=0.60, Levenshtein]
GoogIe → Google LLC [score=0.50, alias]
Google Inc. → Google LLC [score=0.64, Levenshtein]
谷歌 → Google LLC [score=0.00, alias]
國立臺灣大學 → 國立臺灣大學 [score=1.00, Levenshtein]
台灣大學 → 國立臺灣大學 [score=0.50, alias]
台大 → 國立臺灣大學 [score=0.17, alias]
國立台灣大學 → 國立臺灣大學 [score=0.83, Levenshtein]

=====
清理後文章：
=====

根據今日財報分析，台灣積體電路製造股份有限公司 在本季表現亮眼，
而另一家半導體大廠 台灣積體電路製造股份有限公司 也表示產能滿載。
與此同時，鴻海精密工業股份有限公司 與其夥伴 鴻海精密工業股份有限公司 宣布了新的合作計畫，
但在網路論壇上，部分網友卻誤植為 鴻海精密工業股份有限公司。

學術界方面，國立臺灣大學 發表了最新的 AI 研究成果，
參與研究的學生不少來自 國立臺灣大學 與 國立臺灣大學，
這項研究也獲得了美國科技巨頭 Google LLC 的技術支援，
雖然在合約中對方的正式名稱標註為 Google LLC。
```

Observation for Task 2 and Task 3

spaCy 可以快速找出一些人名與地名，但中文文本中如果出現錯字或非標準寫法，例如「頻果」、「蔡英蚊」、「銅羅灣」、「加洲」，模型不一定會辨識成功，也可能給錯標籤。這表示 NER 的輸出通常還需要後處理，才能變成乾淨一致的資料。

Levenshtein normalization 的重點是 threshold 的取捨。threshold 太低時，無關的短詞也可能被硬配到某個標準名稱；threshold 太高時，像 GoogIe、台機電 這種錯字或縮寫又可能抓不到。因此這份實作先用 Levenshtein 分數做主要判斷，再用 alias table 補足縮寫、俗稱、英文正式名稱轉換等情況。

替換時一開始用 `for entity ... cleaned_text.replace(entity, ...)` 的迴圈會踩雷：例如 鴻海精密 已經被換成 鴻海精密工業股份有限公司 之後，下一輪處理 鴻海（長度更短）時，會去 match 剛剛標準名稱開頭的「鴻海」，產生 鴻海精密工業股份有限公司精密工業股份有限公司，Google LLC 也被多加一個 LLC。改用 `re.sub` 配合長度

由長到短的 alternation pattern 一次掃完原文，就只會掃 messy_text 本身、不會回頭再掃替換後的內容，問題就解掉了。